NRLC-05: Synthetic Monitor Migration

> **Series:** NRLC — New Relic to Dynatrace Migration Deep Dives | **Notebook:** 5 of 9 | **Created:** April 2026 | **Last Updated:** 10/14/2026

Overview

New Relic offers four synthetic monitor types plus two specialized types: Ping, Simple Browser, API (multi-step), Step (Scripted Browser), Certificate Check, and Broken Links. Dynatrace offers HTTP, Browser, and Multi-step HTTP monitors. This deep dive maps the NR types to their DT equivalents, covers the location mapping (NR public/private → DT public/private), and flags the scripted-browser caveat that drives most of the manual migration effort.

Phase 24 specialized synthetics (post-2026-04-15): `synthetic_specialized_transformer` now auto-converts **Certificate Check** monitors → DT HTTP Monitor with a certificate-expiry validation rule (the documented rule type is `certificateExpiryDateConstraint` ; see §1 — the transformer emits `certificateExpiryDate`), and **Broken Links** monitors → DT Multi-step HTTP Monitor (capped at 50 links). Ping + Simple Browser + API continue to auto-convert via `synthetic_transformer` (Phase 11). Scripted Browser remains manual.

Table of Contents

1. Synthetic Type Mapping
2. Ping → HTTP Monitor
3. Simple Browser → Browser Monitor
4. API (Multi-step) → Multi-step HTTP Monitor
5. Step (Scripted Browser) → Browser Monitor
6. Location Mapping
7. Authentication & Secrets
8. Validation Pattern

Prerequisites

Requirement	Details
Audience	Engineers migrating synthetic monitors
Standalone	This notebook is self-contained for synthetic migration. No required prerequisite reading.
Optional depth	NRLC-08 (validation), NRLC-09 (toolchain)
NR access	Read scope on Synthetics
DT access	Synthetic create permission; private location IDs if used

Embedded Context — Location & Credential Resolution

Synthetic monitors don't contain NRQL queries, so there's no translation context. The standalone considerations are **location mapping** and **credential re-entry**.

Public location lookup (run at migration time)

```
# Cache DT public location IDs into the transformer's lookup table.
# Classic Environment API v2 endpoint: access token (dt0c01.*) with
# the syntheticLocations.read scope, sent as Api-Token — not a platform token.
curl -H "Authorization: Api-Token $DT_API_TOKEN" \
  https://&lt;tenant&gt;.live.dynatrace.com/api/v2/synthetic/locations
```

Public location IDs are tenant-specific; the transformer queries on first run and caches.

> **Sources:** GET all locations (DT docs) — *"To execute this request, you need an access token with* `syntheticLocations.read` *scope."*

Credential migration checklist

Every synthetic monitor that uses authentication needs its secrets re-entered in DT — not migrated:

Auth Type	NR Storage	DT Storage
Bearer token	Inline in script	DT credentials vault
Basic auth	Inline	DT credentials vault
OAuth client secret	Inline	DT credentials vault
API key in header	Inline	DT credentials vault

The transformer captures the *shape* of the auth (which header, which step) but emits placeholder values.

1. Synthetic Type Mapping

NR Type	DT Type	Transformer	Manual Effort
Ping	HTTP Monitor	`synthetic_transformer` (Phase 11)	None
Simple Browser	Browser Monitor	`synthetic_transformer` (Phase 11)	None
API (Multi-step)	Multi-step HTTP Monitor	`synthetic_transformer` (Phase 11)	Low — verify chained variable extraction
Certificate Check	HTTP Monitor w/ `certificateExpiryDateConstraint` rule	`synthetic_specialized_transformer` (Phase 24)	Rename the rule type: the transformer emits `certificateExpiryDate` (`main` branch, read 10/14/2026), which is not one of the documented values (`patternConstraint` , `regexConstraint` , `httpStatusesList` , `certificateExpiryDateConstraint`) — http_monitor resource (Dynatrace GitHub)
Broken Links	Multi-step HTTP Monitor (50-link cap)	`synthetic_specialized_transformer` (Phase 24)	Low — verify link list discovery
Step (Scripted Browser)	Browser Monitor (clickpath)	Stub + manual	**High — 30–60 min per script**

The first five auto-convert. For Scripted Browser monitors, the engine emits a stub with a TODO and the original NR script attached as a tag for reference.

2. Ping → HTTP Monitor

NR Field	DT Field
`name`	`name`
`uri`	`requests[0].url`
`period` (1, 5, 10, 15, 30, 60 min)	`frequencyMin`
`locations[]`	`locations[]` (mapped per §6)
`verifyName`, `verifySSL`	`requests[0].validation.rules`
`bypassHEADRequest`	`requests[0].requestBody` /method config

Output: a single-request HTTP monitor that runs from the mapped DT locations on the mapped frequency.

3. Simple Browser → Browser Monitor

Same structure as Ping but `type: BROWSER`. The browser engine differences:

Aspect	NR	DT
Browser engine	Chromium-based	Chromium (private locations) / multi-engine (public)
Page load metrics	first-paint, DCL, LCP, etc.	LCP, FCP, TBT, CLS, page load (DT W3C-aligned)
RUM injection	optional	optional (separate from synthetic; DT RUM is its own module)

Migration is structural; the metrics are roughly equivalent but not bit-identical (different timing methodologies). Document this for any dashboard that compared NR/DT browser timings during dual-run.

4. API (Multi-step) → Multi-step HTTP Monitor

NR API monitors are JS scripts with multiple `$http.get/post/...` calls and chained variable extraction. DT Multi-step HTTP monitors use a declarative request chain.

Conversion approach:

1. Parse the NR JS script statement-by-statement
2. Extract each `$http.*` call into a DT request step
3. Map response body extraction (`$http.response.body.json.field`) to DT's body extractor with JSONPath
4. Convert chained variables (e.g., `var token = response.body.token; ...next request uses token`) to DT request variables
5. Flag any non-trivial JS logic (loops, conditionals, custom helpers) for manual review

Confidence:

- HIGH for linear request chains with simple variable extraction
- MEDIUM for chains with conditional logic
- LOW for scripts with loops or external helper modules

5. Step (Scripted Browser) → Browser Monitor

This is the migration's biggest manual cost.

NR Scripted Browsers use a Selenium-style JS DSL with arbitrary code:

```
$browser.get('https://example.com/login')
  .then(() =&gt; $browser.findElement($driver.By.name('username')).sendKeys('user'))
  .then(() =&gt; $browser.findElement($driver.By.name('password')).sendKeys('pass'))
  .then(() =&gt; $browser.findElement($driver.By.name('submit')).click())
  .then(() =&gt; $browser.waitForAndFindElement($driver.By.id('dashboard'), 5000));
```

DT Browser Monitors use a **clickpath** model: a sequence of typed actions (navigate, click, type, validate). Custom JS hooks exist (`pre/postRequestExecutionScript`) but the model is fundamentally different.

Migration approach:

1. Auto-emit a stub DT Browser Monitor with the target URL
2. Attach the original NR script as a tag/note
3. Engineer rebuilds the user journey using DT's clickpath recorder or DSL
4. Validation: confirm the new clickpath catches the same regressions during a 1-week dual-run

Effort estimate: ~30–60 min per scripted browser monitor, depending on complexity.

6. Location Mapping

The transformer maintains a lookup table from NR location codes to DT location IDs:

NR Public Location	DT Public Location	Notes
`AWS_US_EAST_1`	`GEOLOCATION-9999000000000001` (lookup)	DT IDs are tenant-specific
`AWS_EU_WEST_1`	`GEOLOCATION-...`	
`AWS_AP_SOUTHEAST_2`	`GEOLOCATION-...`	
...	...	

Public location IDs are queried via `GET /api/v2/synthetic/locations` at migration time — they're not portable across tenants. The transformer queries the DT tenant on first run and caches the mapping.

Private locations:

NR Private Location	DT Private Location
Minion installation in your VPC	ActiveGate with synthetic capability

Private locations require DT ActiveGate setup (see Dynatrace docs). The transformer maps NR private location names to DT private location names by string match; mismatches require manual lookup.

7. Authentication & Secrets

Credentials never auto-migrate. This includes:

- Bearer tokens / API keys in headers
- Basic auth username + password
- Cookie values
- OAuth client secrets

The transformer captures the **shape** of the auth (which header, which step) but emits placeholders. Engineers re-enter secrets in the DT credentials vault and reference them from the migrated monitor.

Document every secret in the migration's secret-rotation runbook so nothing gets missed during cutover.

8. Validation Pattern

After migration, validate synthetics with this pattern:

1. **Run-level:** confirm the new DT monitor executes successfully on its first scheduled run
2. **Frequency match:** verify DT runs at the same cadence as NR (no gaps, no double-runs)
3. **Location coverage:** confirm all mapped locations are reporting
4. **Failure detection:** intentionally break the target (in a non-prod env) and confirm DT detects the failure within the same window NR would have
5. **SLA continuity:** during dual-run, confirm DT's availability % matches NR's within ±0.5%

Summary

Synthetic migration is mostly mechanical for Ping, Simple Browser, and API monitors. Scripted Browser monitors are the long pole — budget 30–60 min per script. Plan for credential re-entry as a coordinated event, not as part of the auto-import.

Continue to **NRLC-06 SLO & Workload Migration** for service-level migration.

Tooling for Synthetic-Only Migration

```
# 1. Inventory NR synthetic monitors (Ping, Browser, API, Step)
python3 migrate.py migrate --export-only --components synthetics --output ./synthetics-run

# 2. Transform (auto-converts Ping/Browser/API; stubs Step monitors) and write a report — nothing is imported
python3 migrate.py migrate --dry-run --report --components synthetics --output ./synthetics-run

# 3. Import (export → transform → import); writes ./synthetics-run/rollback-manifest.json
python3 migrate.py migrate --components synthetics --output ./synthetics-run

# 4. Re-enter credentials in the DT credentials vault
# 5. Validate via DT UI: confirm first scheduled run executes successfully
```

> **Flags checked against `migrate.py` on the tool's `main` branch (10/05/2026).** There is no `--transform-only`: `--dry-run` runs export and transform, writes `<output>/transformed/dynatrace_config.json`, and skips the import. `--diff` takes effect only on a full or dry run — with `--import-only` it is ignored, and without `--dry-run` it is computed *after* the import. `--import-only` needs `--input <dir>` and does not write a rollback manifest; a full run writes `<output>/rollback-manifest.json`. Run `python3 migrate.py migrate --help` against your checkout before relying on a flag.

`--diff` is left out here because it matches only dashboards and management zones by name — every monitor is reported as `CREATE`, so it cannot find synthetics that already exist. Check for them in the Synthetic app before step 3.

Scripted browser monitors require manual rebuild as DT clickpaths — budget 30–60 min per script.

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source NewRelic-to-Dynatrace-Migration-Utilities, nrql-engine (planned future home: the `dynatrace-dma` Dynatrace Migration Assistant organization), and nrql-translator projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official Dynatrace documentation and New Relic documentation.